

Morpheus Capital Protocol:Post-Audit Technical Report

Live / Deployed Code Notice: This report covered the active, deployed version of the Morpheus Capital Protocol (V7). Code is live on Ethereum Mainnet (L1) with cross-chain components on Arbitrum (L2). All findings are objective technical observations.



Audited by: rubencrxz

Table of Contents

1. [Overview](#)
2. [Architecture & System Map](#)
3. [Contract-by-Contract Analysis](#)
 - 3.1 [RewardPool.sol](#)
 - 3.2 [ChainLinkDataConsumer.sol](#)
 - 3.3 [Distributor.sol](#)
 - 3.4 [DepositPool.sol](#)
 - 3.5 [L1SenderV2.sol](#)
 - 3.6 [L2TokenReceiverV2.sol](#)
4. [Supporting Libraries](#)
 - 4.1 [LinearDistributionIntervalDecrease](#)
 - 4.2 [LockMultiplierMath](#)
 - 4.3 [ReferrerLib](#)

- 4.4 LogExpMath
- 5. Cross-Contract Data Flows
- 6. Reward & Yield Accounting Model
- 7. Virtual Deposit Accounting
- 8. Storage Layout & Upgrade Safety
- 9. Access Control Architecture
- 10. External Dependencies
- 11. Numerical Constants & Precision
- 12. Protocol Invariants as Stated

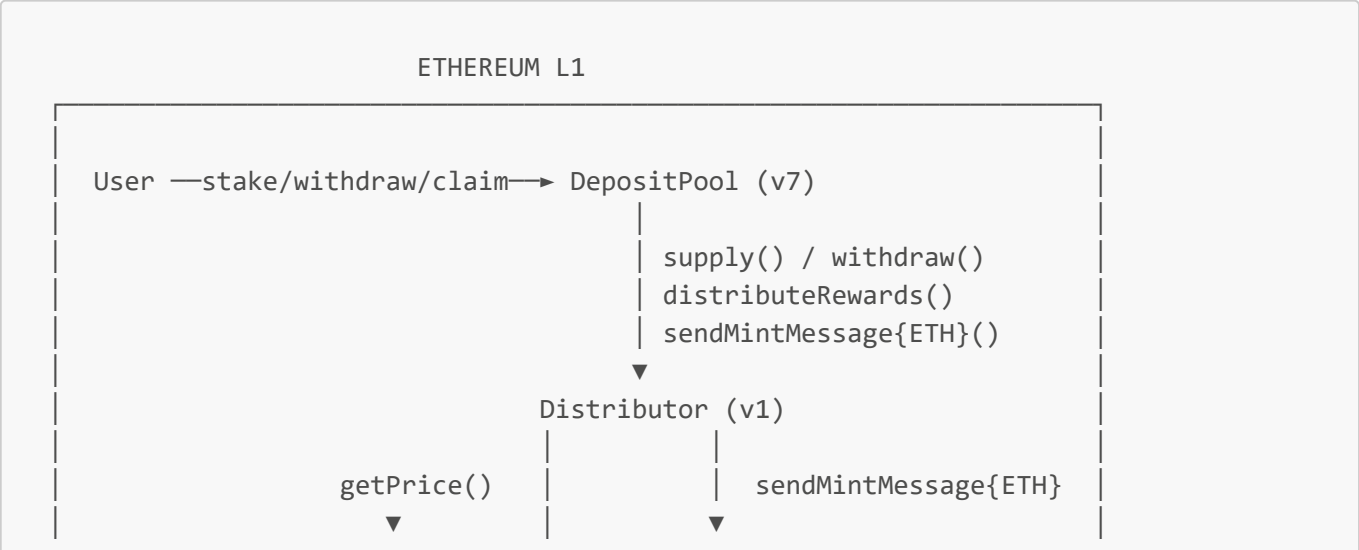
1. Overview

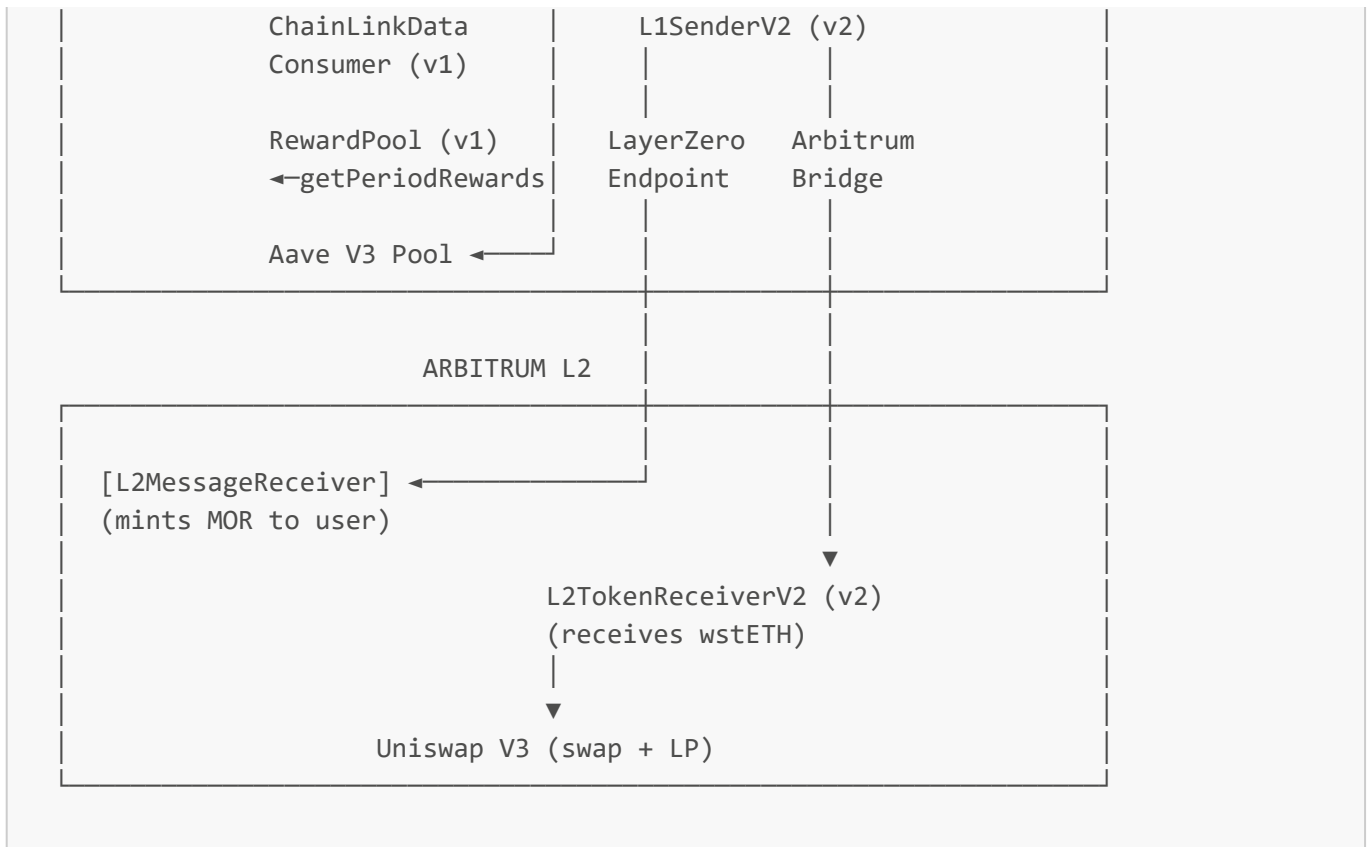
Morpheus is a decentralized AI protocol that incentivizes capital contributions via staking of yield-bearing assets. Stakers receive MOR token rewards proportional to the USD-denominated yield their staked tokens generate. The V7 Capital Protocol is the **live, deployed** iteration, representing a significant architectural change from V6 (**DistributionV6**): it introduces multi-token staking where each stake token has its own **DepositPool** contract, and a central **Distributor** contract orchestrates yield collection and reward allocation across all pools.

Scope: 6 contracts, 390 nSLOC.

Contract	Version	Chain	Role
RewardPool.sol	v1	L1 (Ethereum)	Reward curve definition
ChainLinkDataConsumer.sol	v1	L1 (Ethereum)	Price oracle wrapper
Distributor.sol	v1	L1 (Ethereum)	Yield + reward orchestrator
DepositPool.sol	v7	L1 (Ethereum)	Per-token staking contract
L1SenderV2.sol	v2	L1 (Ethereum)	Cross-chain bridge sender
L2TokenReceiverV2.sol	v2	L2 (Arbitrum)	Liquidity manager

2. Architecture & System Map





Key relationships:

- **DepositPool** calls **Distributor** for all DeFi operations (supply, withdraw, reward distribution, send-mint)
- **Distributor** calls **ChainLinkDataConsumer** for token prices and **RewardPool** for reward amounts
- **Distributor** uses Aave V3 as its yield strategy for staked tokens
- MOR minting messages travel: **Distributor** → **L1SenderV2** → **LayerZero V1** → **L2MessageReceiver**
- Yield (wstETH) travels: **Aave** → **L1SenderV2** → **Arbitrum Bridge** → **L2TokenReceiverV2**

3. Contract-by-Contract Analysis

3.1 RewardPool.sol

Purpose: Defines and exposes the MOR emission schedule for each reward pool.

Storage:

```
RewardPool[] public rewardPools;
```

Each **RewardPool** struct contains:

- **initialReward** — MOR emitted per **decreaseInterval** at protocol start
- **rewardDecrease** — how much **initialReward** decreases per interval
- **payoutStart** — epoch timestamp when emissions begin
- **decreaseInterval** — seconds per emission interval

- `isPublic` — whether it's a public (staker-accessible) or private (admin-managed) pool

Behavior:

- Pools can only be **added**, never edited or removed.
- `onlyExistedRewardPool`, `onlyPublicRewardPool`, `onlyNotPublicRewardPool` are view-only guard functions called by other contracts (they revert, do not return a bool).
- `getPeriodRewards(index_, startTime_, endTime_)` delegates to `LinearDistributionIntervalDecrease.getPeriodReward` using the pool's parameters.
- This is a read-only accumulator; it has no state about what has been distributed — that tracking lives in `Distributor`.

Upgrade model: UUPS, no `removeUpgradeability` guard (unlike `DepositPool`).

3.2 ChainLinkDataConsumer.sol

Purpose: Wraps Chainlink data feeds and supports chained (multi-hop) price computation.

Storage:

```
mapping(bytes32 => address[]) public dataFeeds;
uint64 public allowedPriceUpdateDelay;
```

Path ID encoding: A "path" is an arbitrary string (e.g. "ETH/USD" or "TOKEN/ETH/USD"). Its `bytes32` identifier is:

```
keccak256(abi.encodePacked(path_))
```

Multi-hop price computation (`getChainLinkDataFeedLatestAnswer`):

```
feeds = [aggregator_A, aggregator_B, aggregator_C]

result:
    res = answer_A (raw int256 → uint256)
    baseDecimals = aggregator_A.decimals()
    res = (res * answer_B) / (10 ** aggregator_B.decimals())
    res = (res * answer_C) / (10 ** aggregator_C.decimals())
    return res.convert(baseDecimals_, 18)
```

This design allows computing cross-rates (e.g. $\text{TOKEN/ETH} \times \text{ETH/USD} = \text{TOKEN/USD}$) by chaining Chainlink pairs. The output is always normalized to **18 decimals** via `DecimalsConverter`.

Staleness enforcement:

```

if (answer_ <= 0) return 0;
if (block.timestamp < updatedAt_ || block.timestamp - updatedAt_ >
    allowedPriceUpdateDelay) return 0;

```

The sentinel value `0` is returned on any failure: negative answer, stale data, reverting aggregator (caught in `try/catch`). Callers must handle `0` as a failure signal.

Important detail: `allowedPriceUpdateDelay` defaults to `0` after initialization. With a delay of `0`, *any* price that is not from the current block would fail the staleness check (`block.timestamp - updatedAt_ > 0`). The owner must call `setAllowedPriceUpdateDelay` before price feeds are usable.

Upgrade model: UUPS, owner-only authorization.

3.3 Distributor.sol

Purpose: Central orchestrator. Manages yield collection from Aave, allocates MOR rewards to each `DepositPool` based on USD-weighted yield, and proxies mint messages to `L1Sender`.

Core storage:

```

mapping(uint256 => mapping(address => DepositPool)) public depositPools;
mapping(address => bool) public isDepositTokenAdded;
mapping(uint256 => mapping(address => uint256)) public distributedRewards;
mapping(uint256 => address[]) public depositPoolAddresses;
mapping(uint256 => uint128) public rewardPoolLastCalculatedTimestamp;
uint256 public undistributedRewards;
uint256 public minRewardsDistributePeriod;
address public chainLinkDataConsumer;
address public rewardPool;
address public l1Sender;
address public aavePool;
address public aavePoolDataProvider;
address public aaveRewardsController;

```

`DepositPool` struct (internal):

```

token           - ERC20 staked token (address)
chainLinkPath   - string path for price lookup
deposited       - principal tracked by Distributor
lastUnderlyingBalance - aToken balance at last snapshot
tokenPrice      - last fetched price (18 dec, set in distributeRewards)
strategy        - AAVE | NONE | NO_YIELD
aToken          - corresponding Aave aToken (for AAVE strategy)
isExist         - existence flag

```

addDepositPool flow

Strategy.NO_YIELD:

- Pool must be private (not public)
- Only one deposit pool allowed per reward pool index
- token = address(0), chainLinkPath = ""

Strategy.AAVE:

- Pool must be public
- Token must not already be added (one deposit pool per token)
- Fetches aToken address from AavePoolDataProvider
- Approves token → aavePool (max)
- Approves aToken → aavePool (max)
- Updates prices immediately

distributeRewards(rewardPoolIndex_) — core distribution logic

This is the most critical function. It is public (callable by anyone) but rate-limited.

1. Validate pool exists, lastCalculatedTimestamp != 0
2. rewards_ = RewardPool.getPeriodRewards(index, lastTimestamp, block.timestamp)
→ If rewards_ == 0: return early
3. If private pool:
→ distributedRewards[index][dpAddr] += rewards_
→ rewardPoolLastCalculatedTimestamp[index] = block.timestamp
→ return
4. If public pool:
→ If block.timestamp <= lastTimestamp + minRewardsDistributePeriod: return (throttle)
→ rewardPoolLastCalculatedTimestamp[index] = block.timestamp
5. updateDepositTokensPrices(index):
→ For each deposit pool, fetch latest price from ChainLink
→ require(price > 0) ← reverts if any price is zero
6. Compute yield for each deposit pool:
yieldToken = aToken (AAVE strategy) or token (NONE strategy)
balance = IERC20(yieldToken).balanceOf(address(this))
underlyingYield = (balance - lastUnderlyingBalance).to18(decimals)
yield_i = underlyingYield * tokenPrice
lastUnderlyingBalance = balance ← snapshot update
7. totalYield = $\sum$ yield_i
If totalYield == 0:
undistributedRewards += rewards_
return
8. For each deposit pool with yield_i > 0:
distributedRewards[index][dpAddr] += (yield_i * rewards_) / totalYield

Key observation about yield computation: `yield_` is a product of a normalized quantity (18 dec, via `.to18()`) and a price (18 dec), so each `yield_i` is in `1e36` units. Since it's only used as a proportion (`yield_i / totalYield`), the scale cancels out, and the unit of `distributedRewards` is the same as `rewards_` (raw MOR token units).

undistributedRewards: Accumulates when the protocol earns MOR rewards but has zero yield (e.g., immediately after deployment or when aToken balance hasn't changed). Owner disperses these manually via `withdrawUndistributedRewards`.

supply and withdraw (called by DepositPool only)

Access is enforced via `_onlyExistedDepositPool(rewardPoolIndex_, msg.sender)` — the caller's address must match a registered deposit pool.

supply:

1. Calls `distributeRewards` and `_withdrawYield` (settles state first)
2. Transfers token from DepositPool → Distributor
3. Deposits to Aave: `AaveIPool.supply(token, amount, this, 0)`
4. Updates: `depositPool.deposited += amount, depositPool.lastUnderlyingBalance += amount`

withdraw:

1. Calls `distributeRewards`
2. Caps amount at `depositPool.deposited`
3. Updates `deposited` and `lastUnderlyingBalance` first
4. Calls `_withdrawYield` (sends accumulated yield to L1Sender)
5. Withdraws from Aave back to DepositPool: `AaveIPool.withdraw(token, amount, depositPoolAddress_)`

_withdrawYield

```
yield = lastUnderlyingBalance - deposited
if yield == 0: return

if AAVE: AaveIPool.withdraw(token, yield, l1Sender)
else:    IERC20(token).safeTransfer(l1Sender, yield)

lastUnderlyingBalance -= yield
```

Yield flows directly to `l1Sender`, bypassing the Distributor's token balance.

sendMintMessage (proxy)

Called by any registered `DepositPool`. The check is:

```
_onlyExistedDepositPool(rewardPoolIndex_, _msgSender())
```

Forwards to `IL1SenderV2(l1Sender).sendMintMessage{value: msg.value}(user_, amount_, refundTo_)`.

This design uses a single `l1Sender` as the LayerZero message origin regardless of which deposit pool triggered the claim.

3.4 DepositPool.sol

Purpose: Per-token staking contract. Handles stake, withdraw, claim, lock extension, referrers, and migration from V6.

This is the **most complex** contract in scope (version 7, evolved from Distribution V5).

Storage Layout (including upgrade history gaps)

```
bool    public isNotUpgradeable;
address public depositToken;
address public unusedStorage0;      // formerly: l1Sender
Pool[]  public unusedStorage1;      // formerly: Pool[] reward pools
mapping(uint256 => RewardPoolData)   public rewardPoolsData;
mapping(address => mapping(uint256 => UserData)) public usersData;
uint256 public totalDepositedInPublicPools;
mapping(uint256 => RewardPoolLimits) public unusedStorage2; // formerly: pool
limits
mapping(uint256 => ReferrerTier[])   public referrerTiers;
mapping(address => mapping(uint256 => ReferrerData)) public referrersData;
mapping(uint256 => mapping(address => mapping(address => bool))) public
claimSender;
mapping(uint256 => mapping(address => address))           public
claimReceiver;
bool    public isMigrationOver;
address public distributor;
mapping(uint256 => RewardPoolProtocolDetails) public rewardPoolsProtocolDetails;
```

RewardPoolData (per pool, internal accounting):

- `rate` — cumulative per-unit reward since pool inception
- `lastUpdate` — timestamp of last rate update
- `totalVirtualDeposited` — weighted sum of all virtual deposits (denominator for rate accumulation)

UserData (per user per pool):

- `deposited` — real token amount staked
- `virtualDeposited` — `deposited × totalMultiplier / PRECISION` (used for reward share calculation)

- **rate** — pool rate at last user interaction
- **pendingRewards** — snapshot of accrued rewards before last interaction
- **claimLockStart** — timestamp from which lock multiplier is calculated
- **claimLockEnd** — timestamp until which the user is locked from claiming
- **lastStake** — timestamp of last stake (governs **withdrawLockPeriodAfterStake**)
- **lastClaim** — timestamp of last claim (governs **claimLockPeriodAfterClaim**)
- **referrer** — assigned referrer address

RewardPoolProtocolDetails (per pool, owner-configurable):

- **withdrawLockPeriodAfterStake** — seconds after stake before withdraw is allowed
- **claimLockPeriodAfterStake** — seconds after stake before claim is allowed
- **claimLockPeriodAfterClaim** — cooldown between claims
- **minimalStake** — minimum token amount to maintain a position
- **distributedRewards** — snapshot of total rewards from Distributor that this contract has processed

migrate(rewardPoolIndex_) — V6 → V7 transition

```
Guard: !isMigrationOver
If totalDepositedInPublicPools == 0:
  → isMigrationOver = true immediately (nothing to migrate)
Else:
  → remainder = balance - totalDepositedInPublicPools
  → require(remainder > 0) ← yield must exist
  → IERC20(depositToken).transfer(distributor, remainder) ← sends yield
  → Distributor.supply(rewardPoolIndex_, totalDepositedInPublicPools) ← sends
principal
  → isMigrationOver = true
```

This is a **one-shot, irreversible** operation. After migration, all principal moves from DepositPool's balance into Distributor/Aave. The **remainder** (accumulated stETH yield) goes to **distributor** which the new system tracks as yield.

Note: **migrate** calls **distributor.supply()** which calls **distributeRewards()** internally before accepting the supply. This ensures the state is settled before the principal is handed over.

stake flow

1. Guards: pool exists, pool is public
2. **distributor.distributeRewards(index)** – settle pending rewards globally
3. (**currentPoolRate_**, **rewards_**) = **_getCurrentPoolRate(index)**
4. **_stake(msg.sender, index, amount, currentPoolRate_, claimLockEnd_, referrer_)**
 - a. Guard: **isMigrationOver**
 - b. Resolve **claimLockEnd** (use existing if zero)
 - c. Resolve **referrer** (use existing if zero)
 - d. Balance snapshot before/after **safeTransferFrom** (handles stETH share mechanics)
 - e. **distributor.supply(index, amount)** – deposit principal to Aave

```

    f. require(deposited + amount >= minimalStake)
    g. totalDepositedInPublicPools += amount
    h. pendingRewards = getCurrentUserReward(currentPoolRate, userData)
    i. newMultiplier = lockMultiplier(block.timestamp, claimLockEnd) +
    referralMultiplier - PRECISION
    j. virtualDeposited = (deposited + amount) * newMultiplier / PRECISION
    k. _applyReferrerTier → update referrer's virtualAmountStaked
    l. Update rewardPoolData (rate, totalVirtualDeposited)
    m. Update userData (all fields)
5. rewardPoolsProtocolDetails[index].distributedRewards += rewards_

```

stETH handling: The `balanceBefore/balanceAfter` pattern at step (d) correctly handles stETH's share-based mechanics, where `transferFrom` may not transfer the exact `amount` due to rounding in the share/balance conversion.

claimLockStart reset: Every call to `_stake` sets `userData.claimLockStart = block.timestamp`. This means staking extends the lock period reference point, which directly affects the lock multiplier via `LockMultiplierMath`.

withdraw flow

```

1. Guards: pool exists, pool is public
2. distributor.distributeRewards(index)
3. (currentPoolRate_, rewards_) = _getCurrentPoolRate(index)
4. _withdraw(msg.sender, index, amount, currentPoolRate_)
    a. Guard: isMigrationOver
    b. If amount > deposited: cap at deposited
    c. Check withdrawLockPeriodAfterStake
    d. newDeposited = deposited - amount
    e. Enforce minimalStake (newDeposited == 0 or >= minimalStake)
    f. Capture pendingRewards snapshot
    g. newMultiplier = lockMultiplier(block.timestamp, claimLockEnd) +
    referralMultiplier - PRECISION
    h. virtualDeposited = newDeposited * newMultiplier / PRECISION
    i. _applyReferrerTier → update referrer (old and new referrer logic for
    referrer change case)
    j. Update rewardPoolData and userData
    k. totalDepositedInPublicPools -= amount
    l. distributor.withdraw(index, amount) → Aave.withdraw → transfer to user
5. rewardPoolsProtocolDetails[index].distributedRewards += rewards_

```

Important: In `_withdraw`, `claimLockStart` is reset to `block.timestamp` even on withdrawal, just like staking. This resets the lock multiplier base time.

claim / claimFor flow

```

Guards:
- isMigrationOver

```

```

- block.timestamp > lastStake + claimLockPeriodAfterStake
- block.timestamp > lastClaim + claimLockPeriodAfterClaim
- block.timestamp > claimLockEnd

1. distributor.distributeRewards(index)
2. (currentPoolRate_, rewards_) = _getCurrentPoolRate(index)
3. pendingRewards_ = getCurrentUserReward(currentPoolRate_, userData)
4. require(pendingRewards_ > 0)

5. After claim: RESETS lock state:
   multiplier = getLockPeriodMultiplier(0, 0) + getReferralMultiplier(referrer) -
PRECISION
   virtualDeposited = deposited * newMultiplier / PRECISION
   userData.claimLockStart = 0
   userData.claimLockEnd = 0

6. Update rewardPoolData.totalVirtualDeposited with new virtualDeposited
7. distributor.sendMessage{msg.value}(index, receiver, pendingRewards,
msg.sender)

```

Lock reset on claim: After claiming, `claimLockStart = 0` and `claimLockEnd = 0`. The call to `getLockPeriodMultiplier(0, 0)` returns `PRECISION` (1×) because `start_ >= end_` when both are 0. The user must call `lockClaim` again to re-establish a lock and boost their multiplier.

lockClaim flow

```

Guards:
- isMigrationOver
- pool exists
- claimLockEnd_ > block.timestamp
- user is staked (deposited > 0)
- claimLockEnd_ > userData.claimLockEnd ← can only extend, never shorten

1. distributor.distributeRewards(index)
2. Snapshot pendingRewards
3. Resolve claimLockStart (use userData.claimLockStart if set, else
block.timestamp)
4. Compute new multiplier and virtualDeposited
5. Update rewardPoolData.totalVirtualDeposited
6. Update userData (rate, virtualDeposited, claimLockStart, claimLockEnd)

```

The `claimLockStart` is preserved from the existing value if already set, allowing the full lock duration to count from when the user first established the lock (not from the moment they call `lockClaim` to extend it).

claimFor / claimReferrerTierFor — delegated claiming

```

// claimFor: two paths
if (claimReceiver[index][staker] != address(0)) {

```

```

        receiver_ = claimReceiver[index][staker]; // ignore receiver param, use
        registered
    } else {
        require(claimSender[index][staker][msg.sender], "DS: invalid caller");
    }

```

setClaimReceiver: any staker can register a fixed destination address for their rewards. Once set, ANY caller can trigger **claimFor** and the rewards go to the registered address.

setClaimSender: a staker whitelists specific addresses to call **claimFor** on their behalf.

manageUsersInPrivateRewardPool (owner-only)

Used for private pools where the owner manages user balances manually (no ERC20 token transfer for private pool logic). Calls **_stake** or **_withdraw** based on whether the new amount is higher or lower than the current deposit. For **_stake** in a private pool, the **if (IRewardPool(...).isRewardPoolPublic(...))** branch is skipped, so no actual token transfer occurs.

Rate accumulator _getCurrentPoolRate

```

rewards_ = distributor.getDistributedRewards(index, address(this))
           - rewardPoolsProtocolDetails[index].distributedRewards

if (totalVirtualDeposited == 0) return (poolRate, rewards_)

rate_ = poolRate + (rewards_ * PRECISION) / totalVirtualDeposited

```

The **rewards_** here is the "unsettled" reward: MOR that has been allocated to this deposit pool by the Distributor but not yet distributed per-user. This gap is consumed each time any state-changing function calls **_getCurrentPoolRate** and adds the result to **rewardPoolsProtocolDetails.distributedRewards**.

_getCurrentUserReward

```

deposited_ = (virtualDeposited == 0) ? deposited : virtualDeposited
newRewards_ = (currentPoolRate - userData.rate) * deposited_ / PRECISION
return pendingRewards + newRewards_

```

The **virtualDeposited == 0** fallback exists for migration compatibility: users who existed before V7 had no **virtualDeposited** set. They fall back to raw **deposited** until they interact with the new system.

3.5 L1SenderV2.sol

Purpose: The single outbound gateway from Ethereum to Arbitrum for both MOR mint messages (LayerZero) and yield transfer (wstETH via Arbitrum native bridge).

Storage:

```

address public stETH;
address public distributor;
ArbitrumBridgeConfig public arbitrumBridgeConfig;
LayerZeroConfig public layerZeroConfig;
address public uniswapSwapRouter;

```

ArbitrumBridgeConfig: {wstETH, gateway, receiver} — the L1 Arbitrum bridge gateway router and the L2 receiver. **LayerZeroConfig**: {gateway, receiver, receiverChainId, zroPaymentAddress, adapterParams} — the LayerZero V1 endpoint config.

sendMintMessage

```

Access: only distributor
Encodes: receiverAndSender = abi.encodePacked(config.receiver, address(this))
        payload           = abi.encode(user_, amount_)
Calls: ILayerZeroEndpoint(config.gateway).send{value: msg.value}(
    config.receiverChainId,
    receiverAndSenderAddresses_,
    payload_,
    payable(refundTo_),
    config.zroPaymentAddress,
    config.adapterParams
)

```

The **abi.encodePacked** format for LayerZero V1 is the standard packed encoding of the remote contract address (40 bytes: 20-byte receiver + 20-byte sender).

The **refundTo_** address receives any excess LayerZero fee. This is passed in by the DepositPool as **msg.sender** (the original claimer), ensuring unspent gas refunds go to the user who initiated the claim.

sendWstETH (owner-only)

```

1. All stETH balance → IStETH.wrap() → gets wstETH
2. wstETH balance → IGatewayRouter.outboundTransfer{value: msg.value}(
    wstETH, receiver, amount, gasLimit, maxFeePerGas,
    abi.encode(maxSubmissionCost_, "")
)

```

This is triggered manually by the owner. It wraps all accumulated stETH and bridges the full wstETH balance to L2. There is no partial-send option.

setArbitrumBridgeConfig — approval management

When updating the bridge config, the old approvals are explicitly revoked before setting new ones:

```
if (oldConfig_.wstETH != address(0)) {
    IERC20(stETH).approve(oldConfig_.wstETH, 0);

    IERC20(oldConfig_.wstETH).approve(IGatewayRouter(oldConfig_.gateway).getGateway(oldConfig_.wstETH), 0);
}
```

This is a correct pattern. The new approvals are then set to `type(uint256).max`.

swapExactInputMultihop (owner-only)

Constructs a Uniswap V3 multi-hop path and executes an exact-input swap:

```
path = abi.encodePacked(token0, fee01, token1, fee12, token2, ...)
```

This allows the owner to convert yield tokens held in `L1SenderV2` into stETH before bridging, if the yield was received in a different token.

3.6 L2TokenReceiverV2.sol

Purpose: On Arbitrum L2, receives wstETH, manages Uniswap V3 liquidity for the MOR/wstETH pool.

Storage:

```
address public router;
address public nonfungiblePositionManager;
SwapParams public secondSwapParams;
SwapParams public firstSwapParams; // v2 storage addition
```

`SwapParams: {tokenIn, tokenOut, fee, sqrtPriceLimitX96}`

Initialization note: The `firstSwapParams_` parameter is commented out in `L2TokenReceiver__init`. Only `secondSwapParams` is initialized at deploy time. `firstSwapParams` remains zero until `editParams(_, true)` is called.

Swap (owner-only)

Uses `ISwapRouter.exactInputSingle` (single-hop). The `sqrtPriceLimitX96` from `SwapParams` allows price-limit enforcement.

increaseLiquidityCurrentRange (owner-only)

Adds liquidity to an existing Uniswap V3 position (NFT). The deadline is hardcoded to `block.timestamp`:

```
deadline: block.timestamp
```

This means the transaction must be included in the same block or fail. It offers no tolerance for block inclusion delay, which is by design for liquidity management operations to avoid stale-price LP additions.

collectFees(tokenId_)

```
// No onlyOwner modifier
function collectFees(uint256 tokenId_) external returns (uint256 amount0_, uint256 amount1_) {
```

This function has **no access control**. Any external caller can collect LP fees for any `tokenId_`. The fees are sent to `address(this)` (the contract itself), so this does not represent a loss of funds — the fees are retained within the contract — but the owner does not trigger the collection; anyone can do it.

Approval management (`_addAllowanceUpdateSwapParams`)

For each param set, three max-approvals are established:

- `tokenIn` → `router` (for swaps)
- `tokenIn` → `nonfungiblePositionManager` (for LP)
- `tokenOut` → `nonfungiblePositionManager` (for LP)

The `editParams` function correctly revokes old approvals when `tokenIn` or `tokenOut` changes.

ERC721 Receiver

The contract implements `IERC721Receiver.onERC721Received` returning its own selector, allowing it to receive Uniswap V3 position NFTs from `safeTransferFrom`.

4. Supporting Libraries

4.1 LinearDistributionIntervalDecrease

Computes MOR emissions for a time window `[startTime_, endTime_]` given a piecewise-linear decreasing schedule.

Emission model:

- Emissions are divided into equal-length `interval_` windows.
- In interval `k` (0-indexed), the full-interval emission is: `initialReward - k * decreaseAmount`
- When `decreaseAmount == 0`, emissions are constant forever (`maxEndTime = type(uint128).max`).

Three-part calculation (when the window spans multiple intervals):

```

result = firstPeriodReward          // partial remainder of start interval
      + secondPeriodReward (full)   // complete intervals between start and
end
      + thirdPeriodReward           // partial beginning of end interval

```

`_calculateFullPeriodReward` uses the arithmetic series formula for sum of decreasing rewards:

$$\sum(\text{initialReward} - k \cdot \text{decrease}) \text{ for } k = 0..ip-1$$

$$= \text{initialReward} * ip - \text{decrease} * ip * (ip-1) / 2$$

`_calculateMaxEndTime` uses ceiling division to find when rewards would reach zero:

```

maxIntervals = ⌈initialReward / decreaseAmount⌉
maxEndTime = payoutStart + maxIntervals * interval

```

The library uses `uint128` for time values and `uint256` for amounts, with no SafeMath (Solidity 0.8+ inherent overflow checks apply).

4.2 LockMultiplierMath

Computes a time-weighted lock multiplier using a `tanh`-based curve, creating a non-linear (diminishing returns) incentive for longer locks.

Hardcoded parameters:

```

periodStart = 1721908800 (Jul 25 2024 12:00:00 UTC)
periodEnd   = 2211192000 (Jan 26 2040 12:00:00 UTC)
distributionPeriod = 489,283,200 seconds (~15.5 years)
powerMax = 16.61327546 × 1e18
maxMultiplier = 10.7 × 1e18
minMultiplier = 1.0 × 1e18

```

Formula:

```

endPower   = tanh(2 * (end - periodStart) / distributionPeriod)
startPower = tanh(2 * (start - periodStart) / distributionPeriod)
multiplier = powerMax * (endPower - startPower)
multiplier = clamp(multiplier, 1.0, 10.7) × 1e18
return (multiplier * PRECISION) / 1e18

```

The `tanh` implementation delegates to `LogExpMath.exp`:

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Clamping behavior:

- `end > periodEnd_` → clamped to `periodEnd_`
- `start < periodStart_` → clamped to `periodStart_`
- `start >= end` → returns `PRECISION` (1×)

The multiplier is a function of the **duration** of the lock window relative to the total distribution period, not just the absolute end date. Two users locking for the same number of seconds but at different times in history will get different multipliers (due to the non-linear tanh curve).

4.3 ReferrerLib

Implements the referral system for both stakers (who use a referrer) and referrers (who earn from being referenced).

Staker bonus: Fixed 1% multiplier applied to virtual deposit:

$$\text{REFERRAL_MULTIPLIER} = \text{PRECISION} * 101 / 100$$

Any user with a non-zero referrer address receives this constant 1.01× multiplier on their virtual deposit.

Referrer reward system:

Referrers accumulate rewards proportional to `virtualAmountStaked` — a tier-based amplification of the total amount staked by users who listed them as referrer.

```
amountStaked = Σ(user deposits with this referrer)
multiplier = _getReferrerMultiplier(tiers, amountStaked)
virtualAmountStaked = amountStaked * multiplier / PRECISION
```

Referrer tiers are owner-set per pool and must be strictly increasing in both `amount` and `multiplier`. The multiplier lookup uses a reverse linear scan (highest tier first):

```
for (uint256 i = referrerTiers.length; i > 0; i--) {
    if (amount_ >= referrerTiers[i - 1].amount) {
        return referrerTiers[i - 1].multiplier;
    }
}
return 0; // below lowest tier
```

Important: if `amountStaked` falls below the lowest tier (or no tiers exist), the referrer multiplier is `0` → `virtualAmountStaked = 0` → referrer accrues no rewards.

The rate-based accumulation for referrers uses the same pattern as user rewards:

```
pendingRewards += (currentPoolRate - referrerData.rate) * virtualAmountStaked /
PRECISION
```

Referrer change: When `_applyReferrerTier` is called with a different referrer (`oldReferrer ≠ newReferrer`), both the old and new referrer's `virtualAmountStaked` are updated atomically, adjusting `totalVirtualDeposited` in the pool accordingly.

4.4 LogExpMath

A Balancer-originated fixed-point math library implementing `exp(x)`, `ln(a)`, `log(arg, base)`, and `pow(x, y)` for 18-decimal fixed-point numbers.

`exp(x)` implementation:

- Decomposes `x` into powers of 2 using precomputed constants (`a0...a11`)
- Uses 12-term Taylor series for the remainder
- Intermediate precision uses 20-decimal fixed-point (`ONE_20 = 1e20`)
- First two terms (`a0`, `a1`) stored without decimals to avoid overflow

`ln(a)` implementation:

- Recursion for `a < 1`: `ln(a) = -ln(1/a)`
- Decomposition via precomputed `a_n` constants
- 6-term Taylor series: `ln(a) = 2*(z + z3/3 + z5/5 + ...)` where `z = (a-1)/(a+1)`
- High-precision variant `_ln_36` used for arguments close to 1.0

Bounds:

- `MAX_NATURAL_EXPONENT = 130e18` (corresponds to $\approx 2^{187}$)
- `MIN_NATURAL_EXPONENT = -41e18`
- Arguments outside these bounds cause a revert.

In `LockMultiplierMath._tanh`, the input `x_` is `2 * normalized_time ∈ [0, 2*1e18]`. Well within bounds.

5. Cross-Contract Data Flows

5.1 Staking

```
User calls DepositPool.stake(poolIndex, amount, claimLockEnd, referrer)
|
└─> Distributor.distributeRewards(poolIndex)
```

5.2 Claiming (MOR reward)

5.3 Yield Transfer (stETH → wstETH → L2)

19 / 26

```

└─ IWStETH.wrap(stETHBalance) → wstETH in L1SenderV2
└─ IGatewayRouter.outboundTransfer{value: msg.value}(
    wstETH, L2TokenReceiverV2, amount,
    gasLimit, maxFeePerGas,
    abi.encode(maxSubmissionCost, "")
)
    |
    ▼ [Arbitrum bridge]
    L2TokenReceiverV2 receives wstETH
    Owner calls L2TokenReceiverV2.swap() or .increaseLiquidityCurrentRange()

```

6. Reward & Yield Accounting Model

6.1 Global Reward Schedule

MOR rewards per pool are computed by `RewardPool` using the piecewise-linear decreasing formula:

```
epoch k reward rate = max(initialReward - k * rewardDecrease, 0) per interval
```

The `Distributor.distributeRewards` call integrates this over time and assigns a share of each period's rewards to each `DepositPool` proportional to its USD-weighted yield.

6.2 Yield-Weighted Reward Distribution

```

For period [lastTimestamp, now]:
    rewards = RewardPool.getPeriodRewards(...)    # MOR tokens to distribute

    For each DepositPool i:
        aBalance_i = aToken_i.balanceOf(Distributor)
        yield_i = (aBalance_i - lastUnderlyingBalance_i).to18(decimals_i) *
        tokenPrice_i

        share_i = yield_i / Σ yield_j
        distributedRewards[pool][dpAddr_i] += rewards * share_i

```

Implication: Two different staking tokens that generate the same USD-denominated yield receive equal shares of MOR rewards. A token generating more yield (even if fewer tokens are staked) captures more MOR.

6.3 Per-User Reward Calculation

The protocol uses the standard Synthetix-style per-share accumulator pattern extended with virtual deposits:

```

poolRate_t = poolRate_t-1 + (MOR_rewards_period * PRECISION) /
totalVirtualDeposited

```

```

userReward = (poolRate_current - userRate_snapshot) * userVirtualDeposited /
PRECISION
            + userPendingRewards

```

This pattern is O(1) per user — no loops, no historical iteration.

6.4 Undistributed Rewards

When `totalYield == 0` (no yield generated by any deposit pool during a distribution window), the `rewards_` for that period cannot be proportionally allocated. They accumulate in `undistributedRewards`. The owner can later distribute these to any user/pool address via `withdrawUndistributedRewards`.

This creates an asymmetry: rewards computed for periods of zero yield are not lost but are not automatically credited to stakers. The owner decides how to handle them.

7. Virtual Deposit Accounting

Each user's effective weight in the pool is not their raw deposit but their **virtual deposit**:

```

virtualDeposited = deposited × totalMultiplier / PRECISION

```

where:

```

totalMultiplier = getLockPeriodMultiplier(claimLockStart, claimLockEnd)
                  + getReferralMultiplier(referrer)
                  - PRECISION

getLockPeriodMultiplier → [1.0, 10.7] × PRECISION
getReferralMultiplier   → PRECISION (no referrer) or PRECISION × 1.01

```

Combined multiplier range:

- No lock, no referrer: $1.0 + 1.0 - 1.0 = 1.0\times$
- Max lock, no referrer: $10.7 + 1.0 - 1.0 = 10.7\times$
- No lock, with referrer: $1.0 + 1.01 - 1.0 = 1.01\times$
- Max lock, with referrer: $10.7 + 1.01 - 1.0 = 10.71\times$

The pool-level `totalVirtualDeposited` is the sum of all user `virtualDeposited` values, updated atomically on every stake/withdraw/claim/lockClaim interaction.

Compatibility edge case: When `virtualDeposited == 0` (pre-V7 users), the reward calculation uses raw `deposited` as a fallback:

```

uint256 deposited_ = userData_.virtualDeposited == 0 ? userData_.deposited :
userData_.virtualDeposited;

```

This ensures users who never called `stake` or `lockClaim` in V7 still accrue rewards correctly until their first V7 interaction.

8. Storage Layout & Upgrade Safety

All contracts use UUPS with OpenZeppelin's upgradeable framework. The key storage contract is `DepositPool.sol` which has evolved through 7 versions.

DepositPool Upgrade History Slots

Slot order	Variable	Status in V7
0	<code>isNotUpgradeable</code>	Active
1	<code>depositToken</code>	Active
2	<code>unusedStorage0</code> (formerly <code>l1Sender</code>)	Unused, preserved
3	<code>unusedStorage1</code> (formerly <code>Pool[]</code>)	Unused, preserved
4	<code>rewardPoolsData</code> mapping	Active
5	<code>usersData</code> mapping	Active
6	<code>totalDepositedInPublicPools</code>	Active
7	<code>unusedStorage2</code> (formerly <code>RewardPoolLimits</code>)	Unused, preserved
8	<code>referrerTiers</code>	Active (from V5)
9	<code>referrersData</code>	Active (from V5)
10	<code>claimSender</code>	Active (from V6)
11	<code>claimReceiver</code>	Active (from V6)
12	<code>isMigrationOver</code>	Active (from V7)
13	<code>distributor</code>	Active (from V7)
14	<code>rewardPoolsProtocolDetails</code>	Active (from V7)

Critical: `unusedStorage0`, `unusedStorage1`, and `unusedStorage2` are placeholder variables that preserve slot positions. These must remain in any future upgrade.

`isNotUpgradeable` Guard

`DepositPool` has a finality mechanism:

```
function removeUpgradeability() external onlyOwner {
    isNotUpgradeable = true;
}
function _authorizeUpgrade(address) internal view override onlyOwner {
```

```
require(!isNotUpgradeable, "DS: upgrade isn't available");
}
```

`RewardPool`, `ChainLinkDataConsumer`, `Distributor`, and `L1SenderV2` do **not** have this guard — they are always upgradeable by the owner.

9. Access Control Architecture

Single trusted role: The Morpheus multisig (`owner()`).

Action	Access
Add/configure reward pools	Owner
Add deposit pools	Owner
Set price oracle parameters	Owner
Set bridge configs (LZ, Arbitrum)	Owner
Trigger yield bridge (<code>sendWstETH</code>)	Owner
Execute Uniswap swaps (L1 and L2)	Owner
Manage private pool users	Owner
Set claim lock periods / minimal stake	Owner
Migrate from V6	Owner
Distribute undistributed rewards	Owner
Claim Aave liquidity mining rewards	Owner
Set <code>minRewardsDistributePeriod</code>	Owner
<code>distributeRewards</code>	Anyone (public)
<code>updateDepositTokensPrices</code>	Anyone (public)
<code>stake</code> / <code>withdraw</code> / <code>claim</code>	User (self)
<code>claimFor</code>	Whitelisted sender or anyone (if <code>claimReceiver</code> set)
<code>lockClaim</code>	User (self)
<code>setClaimSender</code> / <code>setClaimReceiver</code>	User (self)
<code>collectFees</code> (L2TokenReceiverV2)	Anyone

`distributeRewards` as a public function means any external actor can trigger reward distribution and price updates. This is by design — it ensures rewards can be distributed even if the owner is inactive.

10. External Dependencies

Protocol	Interface	Usage
Chainlink	<code>AggregatorV3Interface.latestRoundData()</code>	Token price feeds
Aave V3	<code>IPool.supply()</code> , <code>IPool.withdraw()</code> , <code>IPoolDataProvider.getReserveTokensAddresses()</code> , <code>IRewardsController.claimRewards()</code>	Yield generation and aToken tracking
Lido	<code>IWstETH.wrap()</code> , stETH balance mechanics	Yield token wrapping
LayerZero V1	<code>ILayerZeroEndpoint.send()</code>	MOR mint message cross-chain
Arbitrum Bridge	<code>IGatewayRouter.outboundTransfer()</code>	wstETH L1→L2 transfer
Uniswap V3	<code>ISwapRouter.exactInput()</code> , <code>ISwapRouter.exactInputSingle()</code> , <code>INonfungiblePositionManager</code>	Token swaps and LP management
OpenZeppelin	<code>UUPSUpgradeable</code> , <code>OwnableUpgradeable</code> , <code>SafeERC20</code> , <code>Math</code>	Standard primitives
Solarity	<code>PRECISION</code> , <code>DecimalsConverter</code> , <code>Globals</code>	Precision constants and decimal utilities

LayerZero V1 (not V2): The integration uses the legacy V1 interface (`ILayerZeroEndpoint`). The message format — `abi.encodePacked(receiver, sender)` for the remote address — is V1-specific.

Aave aToken accounting: The protocol relies on `aToken.balanceOf(distributor)` rebasing upward over time (as Aave generates interest). The yield is computed as the growth of this balance above `lastUnderlyingBalance`. This is correct for Aave aTokens which are interest-bearing and rebase continuously.

stETH share mechanics: The `stake` function uses a before/after balance check to handle stETH's share-based transfer where the actual transferred amount can differ by 1-2 wei from the requested amount due to rounding.

11. Numerical Constants & Precision

Constant	Value	Source	Used in
<code>PRECISION</code>	<code>1e25</code> (inferred from Solarity)	<code>@solarity/solidity-lib/utils/Globals.sol</code>	All multiplier math
<code>DECIMAL</code> (DepositPool)	<code>1e18</code>	Inline	ReferrerLib, internal

Constant	Value	Source	Used in
<code>DECIMAL</code> (<code>LockMultiplierMath</code>)	<code>1e18</code>	Inline	<code>LockMultiplierMath</code>
<code>periodStart</code>	<code>1721908800</code>	<code>LockMultiplierMath</code>	Lock period start
<code>periodEnd</code>	<code>2211192000</code>	<code>LockMultiplierMath</code>	Lock period end
<code>powerMax</code>	<code>16.61327546 × 1e18</code>	<code>LockMultiplierMath</code>	tanh scaling
<code>maxMultiplier</code>	<code>10.7 × 1e18</code>	<code>LockMultiplierMath</code>	Upper clamp
<code>minMultiplier</code>	<code>1.0 × 1e18</code>	<code>LockMultiplierMath</code>	Lower clamp
<code>REFERRAL_MULTIPLIER</code>	<code>PRECISION × 101/100</code>	<code>ReferrerLib</code>	Staker 1% bonus
<code>MAX_NATURAL_EXPONENT</code>	<code>130e18</code>	<code>LogExpMath</code>	<code>exp()</code> upper bound
<code>MIN_NATURAL_EXPONENT</code>	<code>-41e18</code>	<code>LogExpMath</code>	<code>exp()</code> lower bound

Precision analysis:

The rate accumulator in `_getCurrentPoolRate`:

```
rate_ += (rewards_ * PRECISION) / totalVirtualDeposited
```

With `PRECISION = 1e25`, `rewards_` in raw MOR tokens (18 dec → up to ~1e27 total supply), and `totalVirtualDeposited` in `deposited × multiplier` units, the rate is a per-unit accumulator. User rewards:

```
(rate_delta * virtualDeposited) / PRECISION
```

This recovers the correct MOR amount with 18-decimal precision.

Yield computation precision: `yield_i = underlyingYield_.to18(decimals) * tokenPrice` where `tokenPrice` is 18 dec. The product has `1e36` scaling. Since rewards are distributed proportionally, the absolute scale does not matter. However, the ratio computation:

```
(yield_i * rewards_) / totalYield_
```

is subject to truncation when `yields_i` are small relative to `totalYield_`. Pools with very small yield could receive zero rewards even if some was owed.

12. Protocol Invariants as Stated

The following are the protocol's own declared invariants (from README.md), presented with their implementation basis:

1. The new version uses updated reward calculation mechanisms by supporting additional staking tokens.

- Implemented via one `DepositPool` instance per staking token, orchestrated by a single `Distributor`.

2. The new version must not lock user funds unless explicitly defined by the protocol.

- `withdrawLockPeriodAfterStake` governs time-locks on withdrawal.
- `minimalStake` prevents partial withdrawals that would leave a sub-minimum balance.
- `isMigrationOver` prevents staking/withdrawal before migration is complete.

3. The share of the `DepositPool` in calculations is taken at the current price of the token in relation to USD; exchange rate fluctuations between claim/stake/withdraw are not taken into account.

- `tokenPrice` is snapshotted at the start of each `distributeRewards` call and held constant for that distribution. Prices do not change mid-transaction.

4. The data feeds for Chainlink are selected by the contract owner and are considered trusted.

- `dataFeeds` is set exclusively by `onlyOwner`. The protocol does not validate feed address bytecode or source.

5. The shares of all `DepositPools` must be recalculated when the amount of staked tokens in any deposit pool changes.

- Calling `distributeRewards` (triggered at the start of every stake/withdraw) recalculates yield contributions for all deposit pools of a given reward pool index at that moment.
 - Price snapshots happen simultaneously for all pools in a single `distributeRewards` call, ensuring consistent pricing within a distribution window.
-